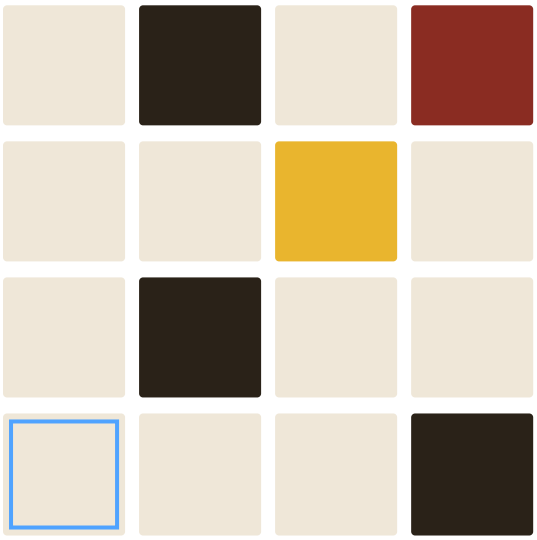
DOCUMENTAÇÃO TÉCNICA

# Wumpus 3D

Como o jogo funciona por dentro, por que cada decisão foi tomada e como reimplementá-lo do zero. Escrito para desenvolvedores júnior.



| | |
|---|---|
| Versão do código | **87ba592** |
| Data | **3 de novembro de 2026** |
| Tecnologias | **TypeScript · Three.js · Vite · Web Workers** |
| Tamanho | **5314 linhas em 31 arquivos de código** |

# Sumário


1 Antes de começar

2 O jogo

3 Tecnologias e decisões de arquitetura

4 Estrutura do projeto e como rodar

5 Regras do jogo — src/game/world.ts

6 A parte 3D — src/render/view.ts

7 Inteligência — base de conhecimento (src/agents/knowledge.ts)

8 Inteligência — os agentes

9 A Arena de agentes (src/arena/)

10 A interface — src/ui, main.ts, index.html e style.css

11 Mapas salvos, exportação e importação

12 Roteiro de reimplementação

13 Como o projeto foi verificado

14 Limitações conhecidas e próximos passos

15 Glossário


Os trechos de código deste documento foram extraídos automaticamente dos arquivos do projeto na versão 87ba592, com a indicação das linhas.

# 1 Antes de começar

Este documento explica o **Wumpus 3D** inteiro, do zero, para quem vai reimplementá-lo. Ele assume que você sabe o básico de JavaScript (variáveis, funções, objetos, arrays) e de HTML/CSS. Todo o resto (TypeScript, Three.js, Web Workers, inferência lógica, probabilidade, algoritmos genéticos) é explicado aqui, no ponto em que aparece.

## Como ler este documento

- **Capítulos 2 a 4** dão a visão geral: o que é o jogo, que tecnologias foram escolhidas e por quê, e como as peças se encaixam. Leia antes de abrir o código.
- **Capítulos 5 a 11** descrevem cada arquivo: o que ele faz, o que implementa, como funciona por dentro, que decisões foram tomadas e o que ele *não* faz.
- **Capítulo 12** é um roteiro de reimplementação em etapas, cada uma com um critério de "pronto".
- **Capítulos 13 e 14** mostram como o projeto foi verificado e quais são as limitações conhecidas.
- O **glossário** no fim explica os termos técnicos.

## Convenções

Os trechos de código são copiados automaticamente dos arquivos do projeto (versão `87ba592`), com o número das linhas. Ao longo do texto aparecem quatro tipos de destaque:

> **DECISÃO**
> Por que algo foi feito de um jeito e não de outro.

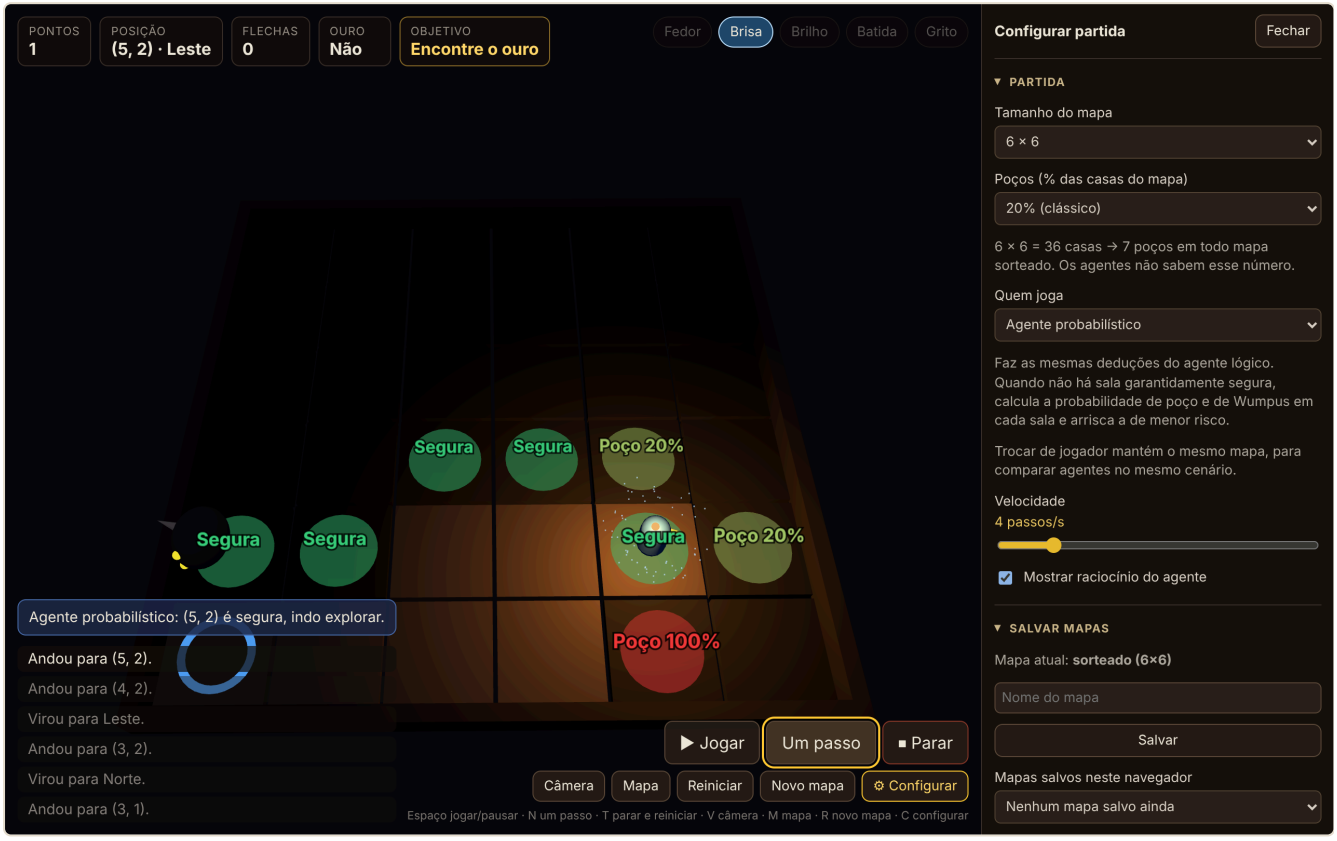
> **O QUE NÃO FAZ**
> Limites intencionais ou conhecidos daquele módulo.

> **ARMADILHA COMUM**
> Erros que um dev júnior costuma cometer ao reimplementar aquela parte.

> **DICA**
> Atalhos e formas de testar.

# 2 O jogo

O **Mundo de Wumpus** é um problema clássico de inteligência artificial, do livro *Artificial Intelligence: A Modern Approach* (Russell e Norvig, o "AIMA"). Ele é usado para ensinar agentes que raciocinam com informação incompleta. Este projeto transforma o problema num jogo 3D no navegador, em que você pode jogar ou assistir agentes jogando.




Tela principal: o agente probabilístico explorando um mapa 6×6. Os discos no piso mostram o que ele acredita sobre cada sala.

## Regras

- A caverna é uma grade de N×N salas (4×4 no clássico; aqui de 4×4 a 10×10). O jogador começa na sala (1, 1), a **entrada**, virado para o leste.
- Cada sala, exceto a entrada, pode ter um **poço** (chance configurável, 20% no clássico). Cair num poço mata.
- Existe um **Wumpus**, um monstro parado numa sala. Entrar na sala dele (vivo) mata. O jogador tem **uma flecha**, que voa em linha reta até a parede e mata o Wumpus se passar pela sala dele.
- Existe um **ouro** numa sala. O objetivo é pegá-lo e **voltar à entrada**. Só isso é vitória: voltar à entrada sem o ouro não encerra a partida.
- O jogador não vê o mapa. Ele só sente **percepções** na sala onde está.

| PERCEPÇÃO | QUANDO ACONTECE |
|---|---|
| Fedor (*stench*) | O Wumpus está numa sala vizinha (ou na mesma). Continua mesmo depois de ele morrer. |
| Brisa (*breeze*) | Existe poço numa sala vizinha. |
| Brilho (*glitter*) | O ouro está nesta sala. |

| PERCEPÇÃO | QUANDO ACONTECE |
|---|---|
| Batida (*bump*) | A última ação foi andar contra a parede. |
| Grito (*scream*) | A flecha acabou de matar o Wumpus. |

"Vizinha" quer dizer as quatro salas ao norte, sul, leste e oeste (não conta diagonal).

## Ações e pontuação

| AÇÃO | O QUE FAZ | PONTOS |
|---|---|---|
| `forward` | Anda uma sala para a frente (ou bate na parede) | −1 |
| `left` / `right` | Gira 90° sem sair do lugar | −1 |
| `shoot` | Atira a flecha na direção em que está virado | −1 e −10; +25 se matar o Wumpus |
| `grab` | Pega o ouro, se ele estiver na sala | −1 e +1000 se pegou |
| (automático) | Chegar à entrada carregando o ouro | +500 e vitória |
| (automático) | Entrar num poço ou na sala do Wumpus vivo | −1000 e derrota |

A partida também termina por **tempo esgotado** depois de `tamanho × tamanho × 20` ações (320 num 4×4). Sem esse limite, um agente sem saída poderia andar para sempre.

> **TRÊS MUDANÇAS EM RELAÇÃO AO LIVRO**
>
> No AIMA existe uma ação `climb` (sair da caverna), e o +1000 só entra ao sair com o ouro. Aqui:
>
> 1. Não existe `climb`. A vitória é automática ao chegar à entrada com o ouro, e voltar sem ele não encerra a partida (pedido do dono do projeto). Por isso os agentes não podem "desistir", e o limite de ações passou a ser necessário.
> 2. Os pontos foram divididos: +1000 ao pegar o ouro e +500 ao voltar com ele, para o jogador ver que pegar o ouro valeu algo.
> 3. Matar o Wumpus dá +25 (no livro, nada). O valor é pequeno de propósito: o tiro certeiro rende +14 líquidos, mas um tiro às cegas da entrada tem valor esperado negativo mesmo no mapa 3×3 (acerta 2 de 8 salas: 25% × 25 = 6,25 < 11), e nenhum desvio arriscado para caçar o Wumpus compensa perto dos 1500 pontos do ouro.

## O que mais o projeto oferece

- **Quatro agentes embutidos**: aleatório, lógico, probabilístico (bayesiano) e genético.
- **Mapas salvos**: salvar no navegador, repetir o mesmo mapa com outro jogador, exportar e importar em JSON.
- **Treino do agente genético** na própria tela, com gráfico.
- **Arena**: outros devs escrevem agentes num arquivo `.js`, importam e competem num ranking que mede pontos, eficiência e qualidade do raciocínio. A mesma arena roda na linha de comando e no GitHub Actions.

# 3 Tecnologias e decisões de arquitetura

## A pilha

| PEÇA | ESCOLHA | PARA QUÊ |
|---|---|---|
| Linguagem | TypeScript | JavaScript com tipos: o editor avisa erros antes de rodar. |
| 3D | Three.js | Biblioteca que esconde a complexidade do WebGL (a API 3D do navegador). |
| Build | Vite | Servidor de desenvolvimento com recarga instantânea e empacotamento para produção. |
| Interface | HTML + CSS + DOM puro | Sem React ou outro framework. |
| Persistência | `localStorage` | Guardar mapas, genoma e agentes no navegador. |
| Hospedagem | GitHub Pages (ou qualquer site estático) | O resultado do build é só HTML, CSS e JS. |

> **POR QUE THREE.JS E NÃO UNITY, GODOT OU BABYLON.JS**
>
> O jogo é uma grade simples com poucos objetos. Unity e Godot exportam para a web, mas geram pacotes de 10 a 40 MB e demoram a carregar. Babylon.js é um motor completo (física, GUI), mais pesado do que o necessário. O Three.js deixa o site com ~160 KB compactados e roda em qualquer navegador moderno.

> **POR QUE TYPESCRIPT E NÃO JAVASCRIPT PURO**
>
> O projeto tem muitos objetos com formato fixo (percepções, decisões, mapas, estatísticas). Com tipos, se você esquecer um campo ou escrever `breze` em vez de `breeze`, o erro aparece no editor. Os agentes externos, por outro lado, são JavaScript puro, para qualquer dev conseguir escrever um sem instalar nada.

> **POR QUE NÃO USAR REACT**
>
> A interface é pequena: placares, um painel e uma tela de Arena. O DOM puro resolve sem dependência extra e sem passo de compilação de JSX. A contrapartida é que a tela é atualizada "à mão" (por exemplo, `updateHud()` em `src/ui/hud.ts`). Se a interface crescer muito, vale migrar.

## Configuração do TypeScript

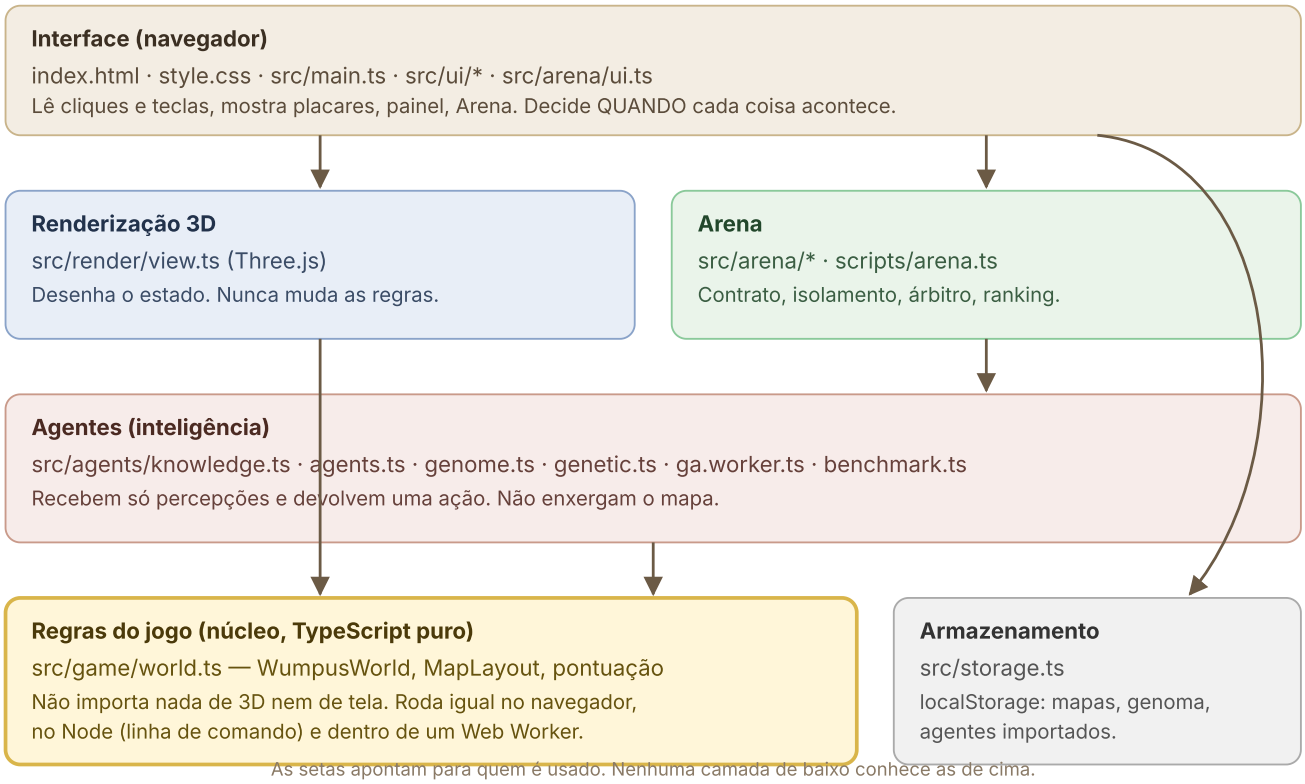
O `tsconfig.json` veio do modelo do Vite e tem três opções que afetam como o código é escrito:

- `noUnusedLocals` e `noUnusedParameters`: variável ou parâmetro sem uso é erro. Mantém o código limpo.
- `erasableSyntaxOnly`: só permite sintaxe TypeScript que some ao tirar os tipos. Na prática: **sem** `enum` (use uniões de strings, como `'won' | 'dead-pit'`) e **sem "parameter properties"** (`constructor(private x: number)`). Por isso as classes declaram campos e atribuem no construtor.
- `verbatimModuleSyntax`: importações só de tipo precisam de `import type` (ou `type` dentro das chaves).

## A regra de ouro da arquitetura

**As regras do jogo não sabem que existe 3D, tela ou agente.** O arquivo `src/game/world.ts` é TypeScript puro: recebe ações e devolve o novo estado. Isso permite:

- rodar milhares de partidas sem desenhar nada (avaliação, treino genético, arena);
- rodar as mesmas regras no Node (linha de comando, GitHub Actions) e num Web Worker;
- trocar o visual inteiro sem tocar nas regras.



# 4 Estrutura do projeto e como rodar

## Rodando

Pré-requisito: Node.js 20 ou superior.

Terminal

```
npm install          # instala dependências (uma vez)
npm run dev          # servidor de desenvolvimento em http://localhost:5173
npm run build        # gera a versão final em dist/ (tsc verifica os tipos antes)
npm run preview      # serve dist/ para testar a versão final
npm run arena        # competição de agentes na linha de comando
```

## Arquivos

| ARQUIVO | LINHAS | RESPONSABILIDADE |
|---|---|---|
| `index.html` | 332 | Toda a estrutura da página: área 3D, placares, botões, painel, tela da Arena. |
| `src/style.css` | 970 | Layout e aparência. Tema escuro único. |
| `src/main.ts` | 140 | Ponto de entrada: liga teclado, botões e painéis aos módulos de `src/ui`. |
| `src/ui/state.ts` | 60 | Estado da partida em andamento e campos de configuração. |
| `src/ui/game.ts` | 185 | Controle da partida: ação, passo do agente, recomeçar, câmera, laço de animação. |
| `src/ui/hud.ts` | 110 | Placares, percepções, pensamento, registro de eventos e fim de jogo. |
| `src/ui/players.ts` | 50 | Quem joga: nomes, opções de "Quem joga" e criação do agente. |
| `src/ui/maps-panel.ts` | 111 | Seção "Salvar mapas". |
| `src/ui/genetic-panel.ts` | 154 | Seção "Agente genético": pesos, gráfico e treino. |
| `src/ui/bench-panel.ts` | 77 | Seção "Avaliação". |
| `src/ui/human-panel.ts` | 125 | Seção "Minhas partidas": árbitro, registro e desafio. |
| `src/ui/dom.ts` | 79 | Utilitários de tela: mensagens, download, copiar, confirmar com dois cliques. |
| `escola.html`, `src/edu/*` | 2929 | Modo escola (D22): Detetive, blocos, criador de cavernas, plano e relatório do professor. Manual próprio. |
| `src/util/format.ts` | 26 | Formatação sem DOM: porcentagens, números, escape de HTML, nomes de arquivo. |
| `src/game/world.ts` | 382 | Regras do jogo, geração e validação de mapas. |
| `src/render/view.ts` | 485 | Cena 3D com Three.js. |
| `src/agents/knowledge.ts` | 167 | Base de conhecimento: deduções lógicas e probabilidades. |

| ARQUIVO | LINHAS | RESPONSABILIDADE |
|---|---|---|
| `src/agents/agents.ts` | 404 | Agentes aleatório, lógico, probabilístico e genético. |
| `src/agents/genome.ts` | 47 | Genes e genoma padrão do agente genético. |
| `src/agents/genetic.ts` | 142 | Algoritmo genético. |
| `src/agents/ga.worker.ts` | 7 | Roda o treino genético em segundo plano. |
| `src/agents/benchmark.ts` | 27 | Partidas rápidas sem 3D (treino, medição e scripts). |
| `src/arena/protocol.ts` | 106 | Contrato com agentes externos. |
| `src/arena/sandbox.ts` | 88 | Isola agentes externos num Web Worker. |
| `src/arena/runner.ts` | 99 | Executa uma partida da arena. |
| `src/arena/referee.ts` | 49 | Árbitro: mede a qualidade do raciocínio. |
| `src/arena/ranking.ts` | 95 | Estatísticas e ordenação. |
| `src/arena/rng.ts` | 26 | Sorteio reproduzível por semente. |
| `src/arena/ui.ts` | 353 | Tela da Arena. |
| `src/report/report.ts` | 191 | Relatórios: modelo de dados, leitura e séries acumuladas. |
| `src/report/charts.ts` | 179 | Gráficos SVG dos relatórios e dicas ao passar o mouse. |
| `src/report/html.ts` | 165 | Tabelas e gráficos em HTML; arquivo de relatório exportado. |
| `src/report/human.ts` | 132 | Partidas humanas: o que conta, desafio por semente, relatório e comparação com os agentes. |
| `src/report/ui.ts` | 118 | Tela de Análise de relatórios e downloads. |
| `src/storage.ts` | 125 | Leitura e escrita no `localStorage`. |
| `scripts/arena.ts`, `scripts/args.ts` | 198 | Arena na linha de comando e leitura das opções. |
| `scripts/treinar.ts`, `scripts/medir.ts` | 124 | Treino do agente genético e tabela de desempenho na linha de comando. |
| `agents/exemplo-cauteloso.js` | 110 | Agente de exemplo para outros devs. |
| `vite.config.ts` | 11 | Configuração do build. |
| `tests/*.test.ts` | 769 | Testes automatizados (Vitest). |
| `scripts/docs/` | — | Geradores destes PDFs. |
| `CLAUDE.md`, `.claude/`, `docs/contexto/` | — | Memória do projeto para o Claude (resumo, skills, subagente, histórico e decisões). |
| `.github/workflows/*.yml` | 72 | Publicação no GitHub Pages e arena em pull requests. |

## Configuração do Vite

vite.config.ts linhas 1–11

```
import { defineConfig } from 'vite';

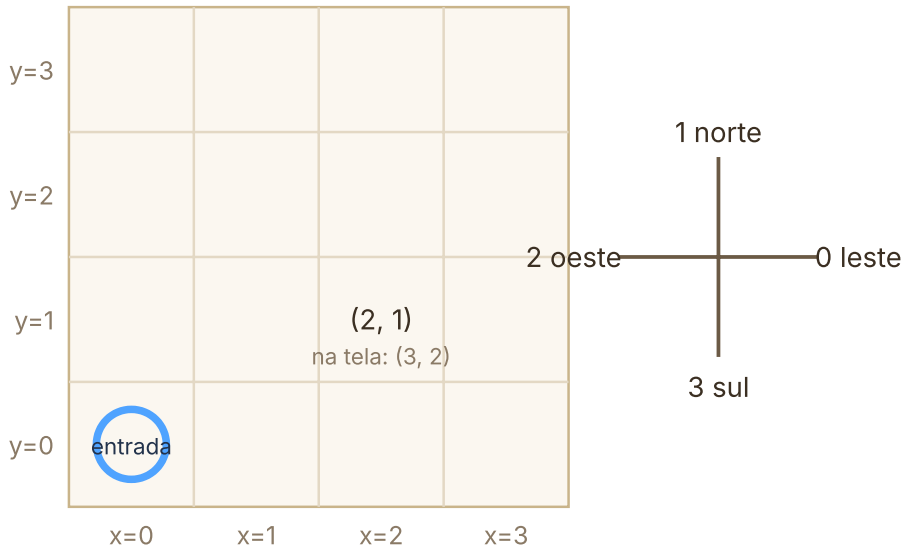
export default defineConfig({
  // Caminhos relativos: funciona no GitHub Pages (em /nome-do-repo/), Netlify, Vercel etc.
  base: './',
  build: {
    chunkSizeWarningLimit: 900, // o modo escola carrega o Blockly (~800 KB) só na página dele
    // Duas páginas: o jogo completo e o modo escola (D22)
    rollupOptions: { input: { main: 'index.html', escola: 'escola.html' } },
  },
});
```

`base: './'` faz o HTML gerado usar caminhos relativos ( `./assets/...` ). Sem isso, no GitHub Pages o site fica em `/nome-do-repo/` e os arquivos não carregam.

# 5 Regras do jogo — src/game/world.ts

O coração do projeto. Tudo o que é regra está aqui, e nada mais. `src/game/world.ts` (382 linhas)

## Coordenadas e direções




No código e na API dos agentes: começa em 0.
Na tela e nos arquivos de mapa (.json): começa em 1.
No 3D: x vira o eixo X e y vira −Z (o norte "entra" na tela).

src/game/world.ts — linhas 4–11

```
export type Dir = 0 | 1 | 2 | 3; // 0 = leste, 1 = norte, 2 = oeste, 3 = sul
export const DIR_VEC: Record<Dir, [number, number]> = {
  0: [1, 0],
  1: [0, 1],
  2: [-1, 0],
  3: [0, -1],
};
export const DIR_NAME: Record<Dir, string> = { 0: 'Leste', 1: 'Norte', 2: 'Oeste', 3: 'Sul' };
```

**ARMADILHA COMUM**

Misturar as duas numerações. O código, o `key()` e a API dos agentes usam **0** para a entrada. A tela e o arquivo de mapa usam **1**. A conversão acontece só em três lugares: nos textos de mensagem ( `+ 1` ), em `toLayout()` e no construtor quando recebe um `layout` .

## Tipos principais

src/game/world.ts — linhas 13–56

```
export type Action = 'forward' | 'left' | 'right' | 'shoot' | 'grab';
export type Status = 'playing' | 'won' | 'dead-pit' | 'dead-wumpus' | 'timeout';

export interface Cell {
  x: number;
  y: number;
}

export interface Percepts {
  stench: boolean; // fedor: Wumpus em sala adjacente (ou na mesma)
  breeze: boolean; // brisa: poço em sala adjacente
  glitter: boolean; // brilho: ouro nesta sala
  bump: boolean; // batida: andou contra a parede
  scream: boolean; // grito: a flecha matou o Wumpus
}

export interface StepResult {
  percepts: Percepts;
  message: string;
  arrow?: { from: Cell; to: Cell; hit: boolean };
}

export const SCORE = {
  action: -1,
  shoot: -10,
  // Bônus pequeno de propósito: dá lucro (+14 líquido do tiro), mas chutar às cegas segue com
  // valor esperado negativo mesmo no 3×3 (25% × 25 < 11) e não compensa desviar do ouro. Ver D14.
  wumpusKill: 25,
  death: -1000,
  gold: 1000, // ao pegar o ouro
  escape: 500, // bônus de vitória: voltar à entrada com o ouro
};

/** Limite de ações por partida, para que nenhum jogo dure para sempre. */
export const maxActionsFor = (size: number) => size * size * 20;

/**
 * Quantos poços um mapa sorteado tem: a porcentagem vale sobre o total de casas, arredondada
 * (20% de 16 casas = 3 poços no 4×4). Pelo menos 1, e sempre sobra espaço para a entrada e o ouro. Ver
D15.
 */
export const pitCountFor = (size: number, pitProbability: number) =>
  pitProbability <= 0 ? 0 : Math.min(size * size - 3, Math.max(1, Math.round(pitProbability * size *
size)));

export const key = (x: number, y: number) => `${x},${y}`;
```

`key(x, y)` transforma uma posição na string `"x,y"`. Ela é usada como chave de `Set` e `Map`, porque em JavaScript dois objetos `{x: 1, y: 2}` diferentes nunca são "iguais" para um `Set`, mas duas strings `"1,2"` são.

## Gerando um mapa

O construtor chama `randomPlacement()`, que sorteia um mapa até achar um em que **seja possível vencer**:

src/game/world.ts — randomPlacement() linhas 159–179

```
function randomPlacement(size: number, pitCount: number, rng: () => number): Placement {
  const free: Cell[] = [];
  for (let x = 0; x < size; x++) {
    for (let y = 0; y < size; y++) {
      if (x !== 0 || y !== 0) free.push({ x, y }); // sala inicial é sempre segura
    }
  }
  const pick = (from: Cell[]) => from[Math.floor(rng() * from.length)];
  for (;;) {
    // Embaralhamento parcial (Fisher-Yates): as primeiras `pitCount` salas viram poços
    const shuffled = [...free];
    for (let i = 0; i < pitCount; i++) {
      const j = i + Math.floor(rng() * (shuffled.length - i));
      [shuffled[i], shuffled[j]] = [shuffled[j], shuffled[i]];
    }
    const pits = new Set(shuffled.slice(0, pitCount).map((c) => key(c.x, c.y)));
    // Wumpus e ouro nunca ficam num poço; os dois podem dividir a mesma sala (aí é preciso matar o
Wumpus). Ver D16.
    const noPit = free.filter((c) => !pits.has(key(c.x, c.y)));
    const wumpus = pick(noPit);
    const gold = pick(noPit);
    if (reachable(size, pits, gold)) return { pits, wumpus, gold };
```

1. O número de poços é fixo: `pitCountFor(tamanho, porcentagem)` = porcentagem × total de casas, arredondado (3 no 4×4 com 20%), com mínimo de 1. Eles vão para salas sorteadas sem repetição (embaralhamento de Fisher–Yates parcial), nunca na entrada.
2. O Wumpus vai para uma sala sem poço, fora da entrada.
3. O ouro vai para uma sala sem poço, fora da entrada. Pode ser a mesma sala do Wumpus: aí é obrigatório matá-lo antes de pegar o ouro.
4. Uma busca em largura (BFS, explicada no capítulo 7) confere se existe caminho da entrada até o ouro sem pisar em poço. O Wumpus não bloqueia, porque pode ser morto com a flecha.
5. Se alguma condição falha, sorteia de novo.

> **POR QUE SORTEAR DE NOVO EM VEZ DE ACEITAR QUALQUER MAPA**
>
> Com as regras do livro, cerca de 26% dos mapas 4×4 eram impossíveis de vencer (ouro dentro de poço, junto do Wumpus ou cercado por poços). Medimos isso em 10.000 mapas. Para um jogo isso é frustrante, então só mapas vencíveis são aceitos. Repare que isso muda levemente as probabilidades reais em relação ao que os agentes supõem (veja o capítulo 14).

> **POR QUE UM NÚMERO FIXO DE POÇOS**
>
> Com uma chance independente por sala, cerca de 4% dos mapas 4×4 saíam sem nenhum poço. O dono do projeto pediu que a porcentagem valesse sobre o total de casas, com pelo menos 3 poços no 4×4. O número exato **não é passado aos agentes**: eles recebem só a porcentagem e continuam supondo poços independentes com essa chance.

O parâmetro `rng` é a função de sorteio. Por padrão é `Math.random`, mas a Arena passa um sorteador com semente, para gerar sempre os mesmos mapas (capítulo 9).

## Um passo do jogo: `step()`

`step(action)` é a única forma de mudar o estado. Ele cobra o ponto da ação, passa a ação para um método próprio ( `turn` , `moveForward` , `shoot` , `grab` ) e recalcula as percepções. Veja `moveForward()` , que concentra as regras de morte e de vitória:

src/game/world.ts — moveForward(), die() e arriveAtEntrance() linhas 321–348

```
private moveForward(): Outcome {
  const a = this.agent;
  const [dx, dy] = DIR_VEC[a.dir];
  const next = { x: a.x + dx, y: a.y + dy };
  if (!this.inBounds(next.x, next.y)) return { message: 'Bateu na parede!', bump: true };

  a.x = next.x;
  a.y = next.y;
  this.visited.add(key(next.x, next.y));
  if (this.hasPit(next.x, next.y)) return this.die('dead-pit', 'Você caiu em um poço! Fim de jogo.');
  if (this.wumpusAt(next)) return this.die('dead-wumpus', 'O Wumpus te devorou! Fim de jogo.');
  if (next.x === 0 && next.y === 0) return this.arriveAtEntrance();
  return { message: `Andou para (${next.x + 1}, ${next.y + 1}).` };
}

private die(status: 'dead-pit' | 'dead-wumpus', message: string): Outcome {
  this.status = status;
  this.score += SCORE.death;
  return { message };
}

/** Vitória só ao voltar à entrada com o ouro; sem ele a partida continua (D6). */
private arriveAtEntrance(): Outcome {
  if (!this.hasGold) return { message: 'De volta à entrada, mas sem o ouro. A partida continua.' };
  this.score += SCORE.escape;
  this.status = 'won';
  return { message: `Você voltou à entrada com o ouro! Vitória! (+${SCORE.escape})` };
}
```

O tiro percorre a linha até a parede e para no Wumpus se o encontrar. O resultado traz `arrow` com origem e destino, para a parte 3D animar a flecha. No fim de todo passo, o limite de ações é conferido.

> **O QUE WORLD.TS NÃO FAZ**
>
> - Não desenha nada e não lê teclado.
> - Não sabe quem está jogando (pessoa ou agente).
> - Não guarda histórico de ações (só o contador `actions` ).
> - Não esconde nada: quem tem o objeto `WumpusWorld` vê o mapa inteiro. Por isso os agentes nunca recebem o objeto, só as percepções.

## Mapas salvos: `MapLayout`

Um mapa é descrito por um objeto simples, que vira JSON para salvar, exportar e importar:

Exemplo de arquivo de mapa (coordenadas começando em 1)

```
{
  "format": "wumpus3d-map",
  "version": 1,
  "name": "Caverna da prova",
  "size": 4,
  "pitProbability": 0.2,
  "pits": [[3, 1], [3, 3], [4, 4]],
  "wumpus": [1, 3],
  "gold": [2, 3]
}
```

`toLayout()` gera esse objeto a partir de uma partida, usando a posição **inicial** do ouro ( `goldStart` ), já que `gold` vira `null` quando é pego. `WumpusWorld.fromLayout()` faz o caminho inverso.

`parseLayout(json)` valida um JSON vindo de fora e lança um `Error` com uma mensagem em português explicando o problema:

src/game/world.ts linhas 91–96

```
function parseLayoutCell(c: unknown, what: string, size: number): [number, number] {
  if (!Array.isArray(c) || c.length !== 2 || !c.every((v) => Number.isInteger(v) && v >= 1 && v <=
size)) {
    throw new Error(`${what} precisa ser [coluna, linha] com valores de 1 a ${size}.`);
  }
  if (c[0] === 1 && c[1] === 1) throw new Error(`${what} não pode ficar na entrada (1, 1).`);
  return [c[0], c[1]];
```

> **ARMADILHA COMUM**
>
> Confiar no JSON importado. Um arquivo pode ter qualquer coisa: tamanho 1000, coordenadas negativas, texto no lugar de números. Valide tudo antes de criar o mundo. `parseLayout` também é usado ao ler o `localStorage` , porque o armazenamento pode ter sido alterado ou corrompido.

# 6 A parte 3D — src/render/view.ts

A classe `WorldView` lê o estado de um `WumpusWorld` e o desenha com Three.js. Ela nunca muda as regras. `src/render/view.ts` (485 linhas)

## Conceitos de Three.js que você precisa

| CONCEITO | O QUE É |
|---|---|
| `Scene` | O "palco": a lista de tudo que existe no mundo 3D. |
| `PerspectiveCamera` | De onde e para onde olhamos. |
| `WebGLRenderer` | Desenha a cena vista pela câmera num `<canvas>`. |
| `Mesh` | Um objeto visível = geometria (forma) + material (cor, brilho). |
| `Group` | Junta vários objetos para mover ou apagar todos juntos. |
| Luzes | `HemisphereLight` ilumina tudo de leve; `PointLight` é uma lâmpada num ponto (a tocha do agente, o brilho do ouro). |
| `Fog` | Névoa: objetos distantes somem no escuro, o que dá clima de caverna. |
| `Sprite` | Imagem 2D que sempre fica de frente para a câmera (usada nos rótulos "Poço 20%"). |

## Do grid para o 3D

src/render/view.ts — linhas 7–12

```
const TILE = 2; // tamanho de cada sala em unidades 3D

// Converte coordenada do grid (x, y) para posição 3D (o norte aponta para -z)
const toWorld = (x: number, y: number) => new THREE.Vector3(x * TILE, 0, -y * TILE);
// Ângulo (yaw) do agente para cada direção: leste, norte, oeste, sul
const DIR_YAW = [-Math.PI / 2, 0, Math.PI / 2, Math.PI];
```

Cada sala ocupa 2×2 unidades. O norte do jogo vira o eixo −Z do Three.js, porque a câmera padrão olha para −Z: assim o norte fica "para dentro" da tela.

## O ciclo de vida da cena

1. **`constructor(container)`**: cria renderizador, cena, névoa e luz ambiente. Usa um `ResizeObserver` para ajustar o tamanho do canvas sempre que a área do mapa muda (a janela ou o painel lateral abrindo e fechando).
2. **`load(world)`**: apaga tudo e constrói a caverna: piso, paredes, poços, Wumpus, ouro, agente e partículas. É chamado a cada partida nova.
3. **`sync()`**: depois de cada ação, decide o que está visível. Salas não visitadas ficam escuras; poços, Wumpus e ouro só aparecem quando a sala foi visitada (ou no fim do jogo, ou com "Mapa" ligado).
4. **`frame()`**: chamado cerca de 60 vezes por segundo. Só anima: aproxima o agente da sala atual, gira, faz a tocha tremular, gira o ouro, move partículas, anda com a flecha e com a câmera.

## Movimento suave sem mexer nas regras

O estado do jogo muda de uma vez (o agente "teleporta" de uma sala para a outra). O 3D esconde isso com interpolação: a cada frame, o modelo anda uma fração da distância até onde deveria estar.

src/render/view.ts linhas 456–463

```
private animateAgent(dt: number, time: number) {
  const a = this.world.agent;
  this.agent.position.lerp(toWorld(a.x, a.y), 1 - Math.exp(-dt * 12));
  let goal = DIR_YAW[a.dir];
  while (goal - this.agentYaw > Math.PI) goal -= Math.PI * 2;
  while (goal - this.agentYaw < -Math.PI) goal += Math.PI * 2;
  this.agentYaw += (goal - this.agentYaw) * (1 - Math.exp(-dt * 14));
  this.agent.rotation.y = this.agentYaw;
```

`1 - Math.exp(-dt * 12)` dá uma fração que não depende da taxa de quadros: num computador rápido ou lento, o movimento leva o mesmo tempo. As linhas com `while` escolhem o caminho de giro mais curto (girar 90° em vez de 270°).

## Mostrando o raciocínio do agente

`showBeliefs(mapa)` recebe, para cada sala, a probabilidade de poço e de Wumpus, e desenha um disco colorido (verde = segura, amarelo a vermelho = risco) com um rótulo. O rótulo é texto desenhado num `<canvas>` 2D e usado como textura de um `Sprite`.

**POR QUE A CÂMERA NÃO GIRA COM O AGENTE**

A câmera segue o agente por trás e de cima, mas sempre do lado sul. Se ela girasse junto, "esquerda" e "direita" na tela mudariam a cada giro, e fica difícil se orientar num grid. A tecla V alterna para a visão geral do mapa inteiro.

**O QUE VIEW.TS NÃO FAZ**

- Não usa modelos 3D externos (.glb): tudo é feito com formas básicas (cápsula, esfera, cone, octaedro). Isso evita arquivos extras e mantém o site leve.
- Não tem som nem sombras (sombras foram removidas porque a tocha criava uma mancha preta sob o agente).
- Não libera a memória de vídeo das geometrias antigas ao recomeçar (veja o capítulo 14).

# 7 Inteligência — base de conhecimento (src/agents/knowledge.ts)

A classe `Knowledge` guarda o que o agente **sentiu** e deduz o que dá para saber a partir disso. Ela é usada pelos agentes lógico, probabilístico e genético, e também pelo árbitro da Arena.
`src/agents/knowledge.ts` (167 linhas)

## O que ela guarda

- `visited`: para cada sala visitada, se havia brisa e fedor.
- `wumpusDead`: se já ouviu o grito.
- `noWumpus`: salas por onde uma flecha passou sem acertar (logo, sem Wumpus).

Ela **não** guarda o mapa: é exatamente o que um jogador humano anotaria num papel.

## Dedução 1: sala sem poço

src/agents/knowledge.ts — linhas 82–85

```
/** Sem poço: já visitada, ou vizinha de uma sala visitada sem brisa. */
pitFree(k: string) {
  return this.visited.has(k) || this.neighbors(k).some((n) => this.visited.get(n)?.breeze === false);
}
```

Se eu estive numa sala e *não* senti brisa, nenhuma vizinha dela tem poço. Se eu estive na sala, ela também não tem (eu estaria morto).

## Dedução 2: onde o Wumpus pode estar

src/agents/knowledge.ts — wumpusCandidates() — linhas 91–105

```
wumpusCandidates(): Set<string> {
  if (this.wumpusDead) return new Set();
  const stenchCells = [...this.visited].filter(([, o]) => o.stench).map(([k]) => k);
  const pits = this.certainPits();
  return new Set(
    this.allCells().filter(
      (k) =>
        !this.visited.has(k) &&
        !this.noWumpus.has(k) &&
        !pits.has(k) &&
        !this.neighbors(k).some((n) => this.visited.get(n)?.stench === false) &&
        stenchCells.every((s) => this.neighbors(s).includes(k)),
    ),
  );
}
```

Uma sala só é candidata se for consistente com **tudo** que foi sentido: não visitada, não limpa por flecha, não vizinha de sala sem fedor, vizinha de **todas** as salas com fedor (existe um só Wumpus, então ele precisa explicar todos os fedores) e sem poço deduzido (pela regra do jogo, o Wumpus nunca fica num poço). Se sobrar uma única candidata, o agente

sabe onde o Wumpus está.

## Dedução 3: poço certo

`certainPits()`: se uma sala tem brisa e só **uma** vizinha ainda pode ter poço, o poço está nessa vizinha.

Juntando tudo: `safe(k)` = sem poço **e** fora das candidatas a Wumpus.

## Probabilidade de poço (inferência bayesiana)

Quando nada é garantidamente seguro, o agente probabilístico precisa de números. A ideia, do capítulo 12 do AIMA:

1. Pegue a **fronteira**: salas desconhecidas vizinhas de salas visitadas.
2. Liste todas as combinações possíveis de poços nessas salas (com 3 salas: $2^3 = 8$ combinações).
3. Descarte as combinações que não explicam alguma brisa sentida.
4. Dê a cada combinação um peso: $p^k(1-p)^{n-k}$, em que $k$ é o número de poços nela e $p$ é a chance de poço por sala.
5. A probabilidade de poço numa sala é: soma dos pesos das combinações com poço nela ÷ soma de todos os pesos.

> **EXEMPLO PARA FAZER NO PAPEL**
>
> Você está em (1,1) após visitar (1,1) e (2,1), e sentiu brisa só em (2,1). A fronteira é A = (3,1) e B = (2,2) (a sala (1,2) é segura porque (1,1) não tinha brisa). Com p = 0,2, as combinações que explicam a brisa são: só A (0,2×0,8 = 0,16), só B (0,16) e A e B (0,04). Total = 0,36. P(poço em A) = (0,16 + 0,04) ÷ 0,36 ≈ **56%**. É exatamente o "Poço 56%" que aparece no jogo nessa situação.

src/agents/knowledge.ts — linhas 28–40

```
function enumeratePits(frontier: string[], breezy: string[][], p: number): number[] {
  const idx = new Map(frontier.map((k, i) => [k, i]));
  const masks = breezy.map((ns) => ns.reduce((m, n) => m | (1 << idx.get(n)!), 0));
  const weights = new Array<number>(frontier.length).fill(0);
  let total = 0;
  for (let combo = 0; combo < 1 << frontier.length; combo++) {
    if (!masks.every((m) => (combo & m) !== 0)) continue; // não explica alguma brisa
    let w = 1;
    for (let i = 0; i < frontier.length; i++) w *= combo & (1 << i) ? p : 1 - p;
    total += w;
    for (let i = 0; i < frontier.length; i++) if (combo & (1 << i)) weights[i] += w;
  }
  return weights.map((w) => (total ? w / total : p));
```

O laço usa **máscaras de bits**: cada combinação é um número inteiro, e o bit *i* ligado significa "poço na sala *i* da fronteira". `combo & m` testa se alguma sala vizinha da brisa tem poço naquela combinação.

> **LIMITE DE 16 SALAS NA FRONTEIRA**
>
> O número de combinações dobra a cada sala: 16 salas são 65.536 combinações, 25 seriam 33 milhões. Acima de 16 o código usa uma estimativa simples (1 ÷ número de vizinhas desconhecidas de cada brisa). Isso só acontece em mapas grandes e é uma aproximação, não o valor exato.

## Probabilidade do Wumpus e risco da sala

O Wumpus é sorteado entre as salas **sem poço**. Por isso, entre as candidatas, a chance de ele estar numa sala é proporcional à chance de ela *não* ter poço: P(Wumpus em c) = (1 − P(poço em c)) ÷ soma de (1 − P(poço)) das candidatas. Com as candidatas igualmente arriscadas, isso dá 1 ÷ número de candidatas.

O risco de entrar numa sala é a chance de morrer lá: `riskOf(b) = P(poço) + P(Wumpus)`. Os dois eventos se **excluem** (nunca estão na mesma sala), então as probabilidades se somam.

src/agents/knowledge.ts — linhas 14–14

```
export const riskOf = (b: CellBelief) => Math.min(1, b.pit + b.wumpus);
```

**O QUE KNOWLEDGE.TS NÃO FAZ**

- A probabilidade do Wumpus usa as probabilidades de poço sala a sala, sem enumerar poços e Wumpus juntos. É uma aproximação: a correlação entre as salas da fronteira não entra nessa conta.
- Ela não sabe que o gerador só aceita mapas vencíveis nem quantos poços o mapa tem; supõe poços independentes com a porcentagem configurada.
- Ela não lembra a ordem das visitas, só o resultado.

# 8 Inteligência — os agentes

`src/agents/agents.ts` (404 linhas), mais `src/agents/genome.ts` (47 linhas), `src/agents/genetic.ts` (142 linhas), `src/agents/ga.worker.ts` (7 linhas) e `src/agents/benchmark.ts` (27 linhas).

## A interface de um agente embutido

src/agents/agents.ts — linhas 8–34

```
export interface Senses {
  x: number;
  y: number;
  dir: Dir;
  hasArrow: boolean;
  hasGold: boolean;
  percepts: Percepts;
}

export const sensesOf = (w: WumpusWorld): Senses => ({
  x: w.agent.x,
  y: w.agent.y,
  dir: w.agent.dir,
  hasArrow: w.hasArrow,
  hasGold: w.hasGold,
  percepts: w.percepts,
});

export interface Agent {
  /** Pensamento da última decisão, mostrado na tela */
  thought: string;
  decide(s: Senses): Action;
  /** O que o agente acredita sobre as salas não visitadas (para desenhar o raciocínio) */
  beliefs(): Map<string, CellBelief> | null;
}

export type AgentId = 'human' | 'random' | 'logic' | 'bayes' | 'genetic';
```

Um agente recebe `Senses` (posição, direção, se tem flecha e ouro, e as percepções) e devolve uma `Action`. `thought` é uma frase para mostrar na tela, e `beliefs()` alimenta os discos coloridos no piso.

## Busca em largura (BFS): como os agentes acham caminhos

Quase toda decisão vira "ir até uma sala". A BFS explora a partir da posição atual, primeiro todas as salas a 1 passo, depois a 2, e assim por diante, guardando de onde veio cada uma. A primeira sala-alvo encontrada é a mais próxima, e o caminho é reconstruído voltando pelos "de onde veio".

src/agents/agents.ts — route() linhas 257–282

```
/** Busca em largura: menor caminho até qualquer sala de `goals`, pisando só em salas `passable`. */
protected route(from: string, goals: Set<string>, passable: (k: string) => boolean): string[] | null {
  if (goals.size === 0) return null;
  const prev = new Map<string, string | null>([[from, null]]);
  const queue = [from];
  while (queue.length) {
    const cur = queue.shift()!;
    if (goals.has(cur)) {
      const path = [cur];
      for (let p = prev.get(cur); p; p = prev.get(p)) path.unshift(p);
      return path;
    }
    for (const n of this.kb.neighbors(cur)) {
      if (prev.has(n) || (!passable(n) && !goals.has(n))) continue;
      prev.set(n, cur);
      queue.push(n);
    }
  }
  return null;
}

/** Direção do primeiro passo de um caminho. */
protected firstStepDir(s: Senses, path: string[]): Dir {
  const [nx, ny] = parseKey(path[1]);
  return dirTo(s.x, s.y, nx, ny);
}
```

O parâmetro `passable` diz por onde pode passar: só salas visitadas (o mais seguro), ou visitadas e comprovadamente seguras. O alvo em si pode ser uma sala arriscada.

## Agente aleatório

Pega o ouro se sentir brilho; no resto escolhe `forward` (60%), `left` (20%) ou `right` (20%). Serve de base: qualquer agente que se diz inteligente precisa ganhar dele com folga.

## Agentes lógico e probabilístico (`ReasoningAgent`)

Os dois são a mesma classe com `mode` diferente. A cada passo, em ordem de prioridade:

| # | SITUAÇÃO | O QUE FAZ |
|---|---|---|
| 1 | Sente brilho e não tem ouro | `grab` |
| 2 | Tem o ouro | Volta para a entrada pelas salas visitadas (a vitória é automática ao chegar) |
| 3 | Existe sala segura não visitada | Vai até a mais próxima |
| 4 | Tem flecha e no máximo 3 candidatas a Wumpus | Vai até uma sala alinhada com uma candidata, mira e atira |
| 5 | Nenhuma sala segura | Lógico: a sala da fronteira com menos indícios (brisas e fedores ao lado). Probabilístico: a de menor risco P(poço) + P(Wumpus). Empate: a mais próxima |
| 6 | Toda a fronteira é perigo certo | Gira no lugar até o limite de ações |

> **POR QUE O AGENTE LÓGICO TAMBÉM ARRISCA**
>
> Na versão do livro, o agente lógico desiste e sai da caverna quando não há sala segura. Como aqui não existe sair, ficar parado só levaria ao tempo esgotado. Então ele chuta, mas sem usar probabilidades: só conta indícios. Essa é a diferença que separa os dois agentes nos resultados.

Após um tiro, se não ouvir o grito, o agente marca as salas da linha do tiro como livres de Wumpus (`arrowMissed`).

## Lógico × probabilístico: fundamentação e diferença

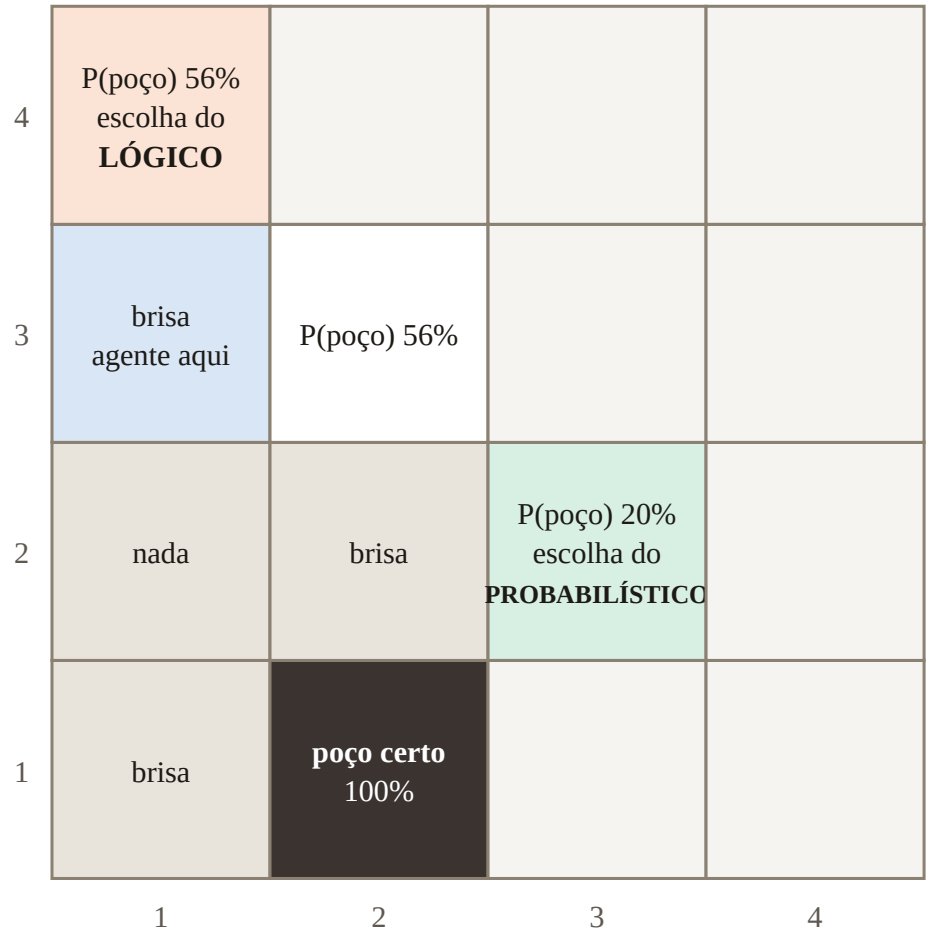
Os dois agentes seguem o mesmo roteiro do **agente híbrido** do capítulo 7 do AIMA: pegar o ouro e voltar; senão, ir para a sala segura mais próxima; senão, tentar matar o Wumpus; senão, arriscar uma sala que não seja perigo comprovado. Eles **sabem exatamente as mesmas coisas**, porque usam a mesma base de conhecimento. A diferença está só no passo 5, quando nenhuma sala é comprovadamente segura.

| | AGENTE LÓGICO | AGENTE PROBABILÍSTICO |
|---|---|---|
| Base teórica | Lógica proposicional e inferência por verificação de modelos (AIMA, cap. 7). | Probabilidade e inferência por enumeração na fronteira (AIMA, cap. 12, seção 12.7), mais a escolha da ação de menor risco. |
| O que conclui sobre uma sala | Três respostas: segura, perigo certo ou "não sei". | Um número: a chance de morrer ao entrar. |
| Passo 5 (sem sala segura) | A lógica não ordena as salas "não sei". O agente usa uma regra prática: a sala com menos brisas e fedores ao redor; empate, a mais próxima. | A sala de menor P(poço) + P(Wumpus); empate, a mais próxima. |
| Ponto forte | Simples e rápido; nunca entra num perigo comprovado. | Usa toda a evidência: percebe quando uma brisa já está "explicada" por outro poço. |
| Ponto fraco | Contar indícios trata todas as brisas como iguais. | Supõe poços independentes com a porcentagem configurada, o que é só uma aproximação do sorteio real. |

**Por que as deduções são completas.** As pistas do jogo viram regras de um formato só: "pelo menos uma vizinha desta brisa tem poço" e "nenhuma vizinha desta sala sem brisa tem poço". Com regras assim, uma sala é poço em **todos** os modelos consistentes só quando é a única vizinha desconhecida de uma brisa, e é livre de poço em todos só quando é vizinha de uma sala sem brisa. As regras de `pitFree` e `certainPits` dão, portanto, o mesmo resultado que a verificação de modelos do livro, sem enumerar modelos.

## Um exemplo real em que eles divergem

A situação abaixo aconteceu no mapa 2 da semente `exemplo` (4×4). O agente visitou (1,1), (1,2), (2,2) e (1,3) e sentiu brisa em (1,1), (2,2) e (1,3). Nenhuma sala desconhecida é segura.




- **(2,1) é poço certo**: a brisa de (1,1) só pode vir dela, porque (1,2) foi visitada.
- **(1,4) e (2,3)**: a brisa de (1,3) precisa de um poço numa delas. Pela enumeração, 56% cada.
- **(3,2)**: é vizinha da brisa de (2,2), mas essa brisa já está **explicada** pelo poço certo em (2,1). Então (3,2) fica com a chance a priori: 20%.
- **O agente lógico** conta 1 indício para (1,4) e 1 para (3,2), desempata pela distância e vai para (1,4): 56% de risco.
- **O agente probabilístico** vai para (3,2): 20% de risco. Esse efeito de "explicar" uma evidência por outra causa é justamente o que a contagem de indícios não capta.

## Correções feitas nesta versão

| O QUE MUDOU | ANTES | POR QUÊ |
|---|---|---|
| O Wumpus nunca é candidato numa sala com poço deduzido | Poço certo podia continuar candidato a Wumpus. | A regra do jogo (D16) proíbe Wumpus em poço; é uma dedução lógica válida para os dois agentes. |
| P(Wumpus) proporcional à chance de a sala não ter poço | 1 ÷ número de candidatas, sempre. | O Wumpus é sorteado entre as salas sem poço. |
| Risco = P(poço) + P(Wumpus) | $1 - (1 - P(\text{poço}))(1 - P(\text{Wumpus}))$, que supõe eventos independentes. | Os dois eventos se excluem; a fórmula antiga subestimava o risco quando os dois eram altos. |
| Empate no passo 5 decidido pela sala mais próxima | Empate decidido pela ordem fixa das salas. | O agente híbrido do livro planeja a rota mais curta; andar menos também custa menos pontos. |

| O QUE MUDOU | ANTES | POR QUÊ |
|---|---|---|
| Comentário do tiro corrigido | Dizia que só o agente lógico atirava sem certeza. | Os dois atiram com até 3 candidatas; o comentário estava errado. |

Efeito medido (mesmos mapas, 5000 no 4×4): o lógico passou de 564 para 570 pontos de média e o probabilístico de 635 para 650. No 6×6 (1000 mapas), de 20 para 46 e de 174 para 182.

## Agente genético

O `GeneticAgent` reaproveita a base de conhecimento e as regras 1 e 2, mas troca as regras 3 a 5 por uma **fórmula de pontuação**. Para cada sala candidata da fronteira, ele calcula 8 características e soma cada uma multiplicada por um peso. As duas primeiras são os termos da pontuação, como numa utilidade esperada: o risco de morrer (custa 1000) e a distância (cada passo custa 1).

src/agents/agents.ts — linhas 373–390

```
private cellFeatures(s: Senses, cell: string, path: string[], safe: boolean, belief: CellBelief):
number[] {
  const kb = this.kb;
  const size = kb.size;
  const [cx, cy] = parseKey(cell);
  const neighbors = kb.neighbors(cell);
  const share = (test: (n: string) => boolean) => neighbors.filter(test).length / 4;
  const diff = (this.firstStepDir(s, path) - s.dir + 4) % 4;
  const turns = diff === 0 ? 0 : diff === 2 ? 1 : 0.5;
  return [
    riskOf(belief),
    (path.length - 1) / size,
    safe ? 1 : 0,
    share((n) => !kb.visited.has(n)),
    (cx + cy) / (2 * size),
    turns,
    share((n) => !!kb.visited.get(n)?.breeze),
    share((n) => !!kb.visited.get(n)?.stench),
  ];
```

O agente vai para a sala de maior nota. Uma segunda fórmula, com 3 pesos (viés, certeza sobre o alvo e "ainda existe sala segura"), dá nota à opção de atirar. Os 11 pesos formam o **genoma**.

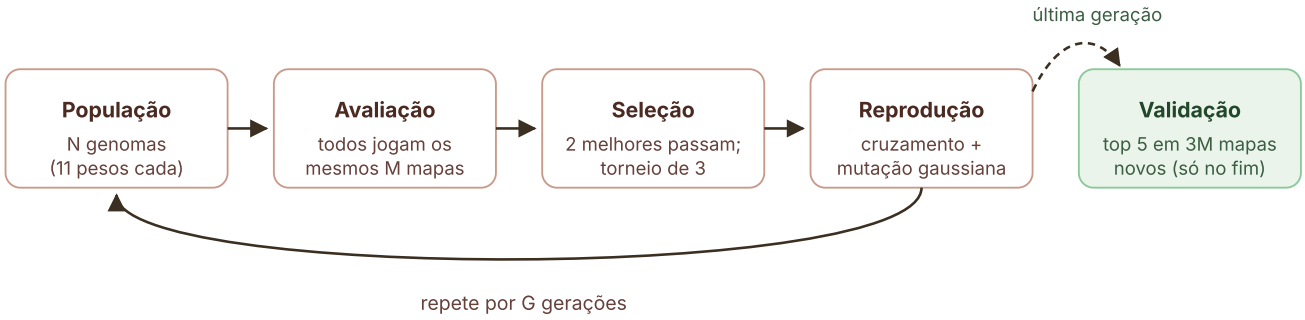
> **POR QUE RISCO E DISTÂNCIA COMO CARACTERÍSTICAS (D19)**
>
> Na versão anterior, o genético pesava P(poço) e P(Wumpus) separadamente e precisava acertar a proporção entre os pesos para chegar à regra do probabilístico. Com o risco pronto, o `BAYES_LIKE_GENOME` (risco −5, distância −0,01) já se comporta como o probabilístico, e o treino parte desse nível para ajustar o resto.

**Compromisso com o alvo.** Depois de escolher uma sala, o agente vai até ela e só refaz a escolha quando aprende algo novo: entrou numa sala nova, ouviu o grito ou errou o tiro. Sem isso, pesos que dependem da posição (distância, giros) mudavam a nota das salas a cada passo, e o agente trocava de alvo, voltava e trocava de novo, até o tempo esgotar. Com pesos treinados assim, uma Arena 6×6 chegou a 53% de tempo esgotado; o teste `tests/genetic.test.ts` garante que isso não volta.

**POR QUE EVOLUIR PESOS E NÃO UMA SEQUÊNCIA DE AÇÕES**

Um algoritmo genético poderia evoluir uma lista fixa de ações ("frente, frente, esquerda...") para um mapa. Mas essa lista só funciona naquele mapa e, na prática, "decora" a resposta. Evoluindo os pesos de uma estratégia, o agente generaliza: joga qualquer mapa sem nunca vê-lo, só pelas percepções, igual aos outros agentes.

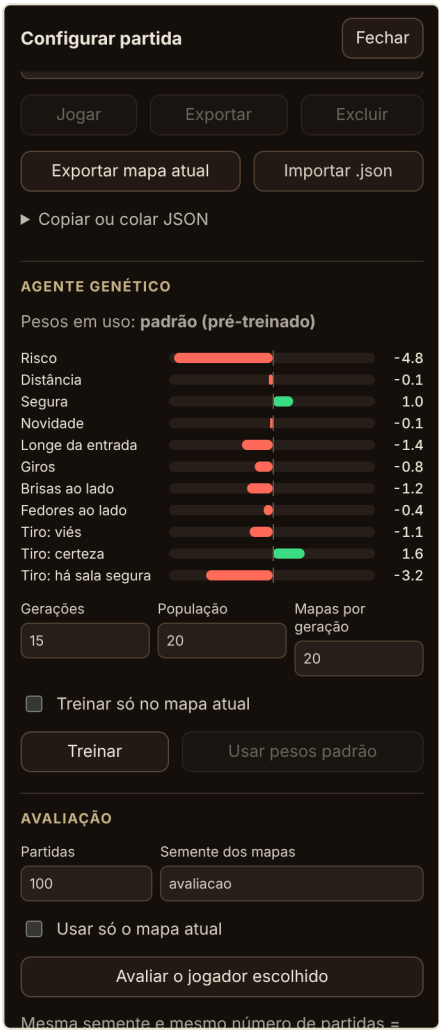
## O algoritmo genético (src/agents/genetic.ts)




1. **População**: N genomas com pesos aleatórios entre −5 e 5. O genoma atual entra na primeira posição, para o treino continuar de onde parou.
2. **Avaliação**: cada genoma joga os **mesmos** M mapas sorteados naquela geração. A **aptidão** (fitness) é a pontuação média.
3. **Elitismo**: os 2 melhores passam direto para a próxima geração.
4. **Seleção por torneio**: para escolher um pai, sorteia 3 genomas e fica com o melhor.
5. **Cruzamento aritmético**: cada peso do filho é um ponto entre os pesos dos pais ( `pai × t + mãe × (1 - t)` , com t sorteado).
6. **Mutação gaussiana**: cada peso tem 20% de chance de receber um ruído de desvio padrão 0,8.
7. **Validação** (só no fim): os 5 melhores jogam 3×M mapas novos, e o vencedor dessa rodada é o resultado.

src/agents/genetic.ts — linhas 104–108

```
if (gen === opts.generations) {
  // Validação: os finalistas jogam mapas que nunca viram
  const winner = validate(scored, sampleMaps(opts.validationMaps
?? opts.mapsPerGeneration * 3));
  yield { generation: gen, best: winner.fit, average:
scored[0].fit, bestGenome: winner.genome, validated: true };
  return;
```




Painel do agente genético: os 11 pesos do genoma padrão (verde = positivo, vermelho = negativo) e os controles de treino.

> **POR QUE A VALIDAÇÃO FINAL EXISTE**
>
> Na primeira versão, o melhor da última geração às vezes era só sortudo: com poucos mapas por geração, um genoma mediano pode ter tido mapas fáceis. O genoma "evoluído" chegou a ficar **pior** que o escrito à mão (725 contra 760 pontos). Validar os finalistas em mapas que eles nunca viram reduz esse efeito. É a mesma ideia de separar dados de treino e de teste em aprendizado de máquina.

O treino roda dentro de `ga.worker.ts`, um **Web Worker**: uma thread separada do navegador. Sem isso, a tela congelaria por segundos a cada geração. O `train()` é um **gerador** (`function*`) que entrega um relatório por geração (`yield`), e o worker repassa cada um para a tela desenhar o gráfico.

Leitura dos pesos: "Poço" −4,6, "Wumpus" −5 e "Segura" +3,4 dizem "fuja do risco, prefira o seguro"; "Tiro: certeza" +3,0 e "Tiro: há sala segura" −2,4 dizem "atire quando tiver certeza de onde está o Wumpus, e menos quando ainda houver sala segura para explorar". O algoritmo chegou sozinho a uma estratégia parecida com a do agente probabilístico.

## Resultados medidos

Mesmos mapas para todos os agentes, 20% de poços (3 no 4×4, 7 no 6×6):

| AGENTE | 4×4 (5000 MAPAS) | 6×6 (1000 MAPAS) |
|---|---|---|
| Aleatório | ~4% de vitórias, −767 pts | ~2%, −854 pts |
| Lógico | 63%, 570 pts | 43%, 46 pts |
| Probabilístico | 67%, 650 pts | 48%, 182 pts |
| Genético (genoma padrão) | 66%, 638 pts | 51%, 242 pts |

Medido com o bônus de +25 por matar o Wumpus, número fixo de poços e o Wumpus fora dos poços. O genético empata com o probabilístico no 4×4 (diferença pareada de −12 pts, intervalo de 95% de −49 a +25) e tende a ficar à frente no 6×6 (+60 pts, intervalo de −18 a +138); nenhuma das diferenças é estatisticamente comprovada. Lotes de 200 a 1000 mapas variam bastante (todos os agentes ficam melhores ou piores juntos), por isso a tabela usa mais mapas.

# 9 A Arena de agentes (src/arena/)

A Arena permite que outros devs escrevam agentes, importem e disputem um ranking justo. Ela tem quatro problemas para resolver: um contrato simples, isolamento contra trapaça, mapas iguais para todos e métricas que meçam mais do que sorte.

## 1. O contrato (protocol.ts)

Um agente externo é um arquivo JavaScript que chama `registerAgent`:

agents/exemplo-cauteloso.js — linhas 9–16

```
registerAgent({
  name: 'Explorador Cauteloso',
  author: 'Equipe Wumpus 3D',
  description: 'Exemplo simples: só pisa onde tem certeza e, sem opção, arrisca o menor perigo.',

  // Chamado no início de cada partida.
  // info = { size, pitProbability, maxActions }
  create(info) {
```

`create(info)` roda no início de cada partida e devolve um objeto com `decide(sentidos)`, chamado a cada passo. `decide` devolve uma ação, ou `{ action, thought, beliefs }` para mostrar o raciocínio. O guia completo para devs está em `AGENTES.md`.

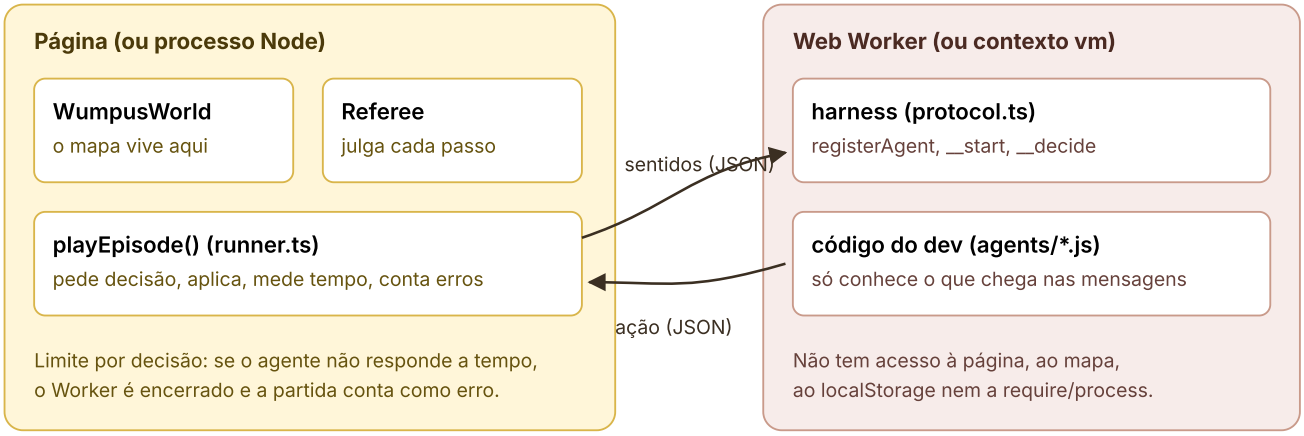
O arquivo do dev é "embrulhado" por um código fixo (o **harness**), que define `registerAgent` antes e as funções `__start` e `__decide` depois:

src/arena/protocol.ts — linhas 86–89

```
export const HARNESS_BEFORE = `
var __defs = [];
function registerAgent(def) { __defs.push(def); }
`;
```

Tudo que o agente devolve passa por `normalizeDecision()`, que confere se a ação existe e limpa o resto (textos cortados em 300 caracteres, probabilidades entre 0 e 1, coordenadas dentro do mapa).

## 2. Isolamento (sandbox.ts e scripts/arena.ts)




**No navegador**, cada agente externo roda num Web Worker criado a partir de um `Blob` (harness + código do dev). O Worker não acessa a página, o DOM nem o `localStorage`. A conversa é por mensagens: a página manda os sentidos, o Worker responde a ação. Cada pedido tem um prazo: se o agente não responder (por exemplo, laço infinito), o Worker é encerrado e recriado na próxima partida.

**No Node**, cada agente roda num contexto `vm` próprio, criado com `Object.create(null)`, e só strings JSON entram e saem. O `vm` aplica o limite de tempo diretamente.

> **POR QUE O MOTOR DO JOGO NÃO RODA JUNTO COM O AGENTE**
>
> Parecia mais simples rodar a partida inteira dentro do Worker do agente. Mas aí o mapa estaria no mesmo ambiente do código do dev, e um agente trapaceiro poderia, por exemplo, substituir `Set.prototype.has` e espionar as consultas que o motor faz ao conjunto de poços. Mantendo o mundo do lado de fora, o agente só conhece o que chega na mensagem.

> **O QUE O ISOLAMENTO NÃO GARANTE**
>
> - O `vm` do Node **não é uma barreira de segurança completa** (a própria documentação do Node avisa). Os truques clássicos para chegar ao `process` foram testados e bloqueados, mas código malicioso sofisticado pode escapar. Por isso o mantenedor precisa revisar o código antes de aceitar um agente no repositório.
> - No navegador o Worker ainda pode usar a rede (`fetch`). Isso não dá acesso ao mapa, mas não é bloqueado.

## 3. Mapas iguais para todos (rng.ts)

`Math.random` não aceita semente: cada execução dá números diferentes. A Arena usa o **mulberry32**, um gerador pequeno que, dada a mesma semente, produz sempre a mesma sequência. A semente pode ser um texto ("final-2026"), transformado em número pelo hash FNV-1a.

src/arena/rng.ts — linhas 23–26

```
export function arenaMaps(seed: string, count: number, size: number, pitProbability: number):
MapLayout[] {
  const rng = mulberry32(seedFromText(seed));
  return Array.from({ length: count }, (_, i) => new WumpusWorld(size, pitProbability,
rng).toLayout(`Arena ${seed} #${i + 1}`));
}
```

Isso foi conferido: a mesma competição no navegador e na linha de comando dá exatamente os mesmos números.

## 4. Medindo o raciocínio (referee.ts)

Pontos dependem muito de sorte (um mapa pode obrigar a arriscar logo no início). O **árbitro** mede a qualidade das decisões. Ele mantém sua própria base de conhecimento, alimentada com as mesmas percepções do agente, e julga cada `forward` *antes* de ele acontecer:

src/arena/referee.ts linhas 33–39

```
const candidates = kb.wumpusCandidates();
if (kb.certainPits().has(target) || (candidates.size === 1 && candidates.has(target))) {
  this.logicErrors++;
} else if (!kb.safe(target, candidates)) {
  const safeLeft = kb.allCells().some((k) => !kb.visited.has(k) && kb.safe(k, candidates));
  if (safeLeft) this.needlessRisks++;
}
```

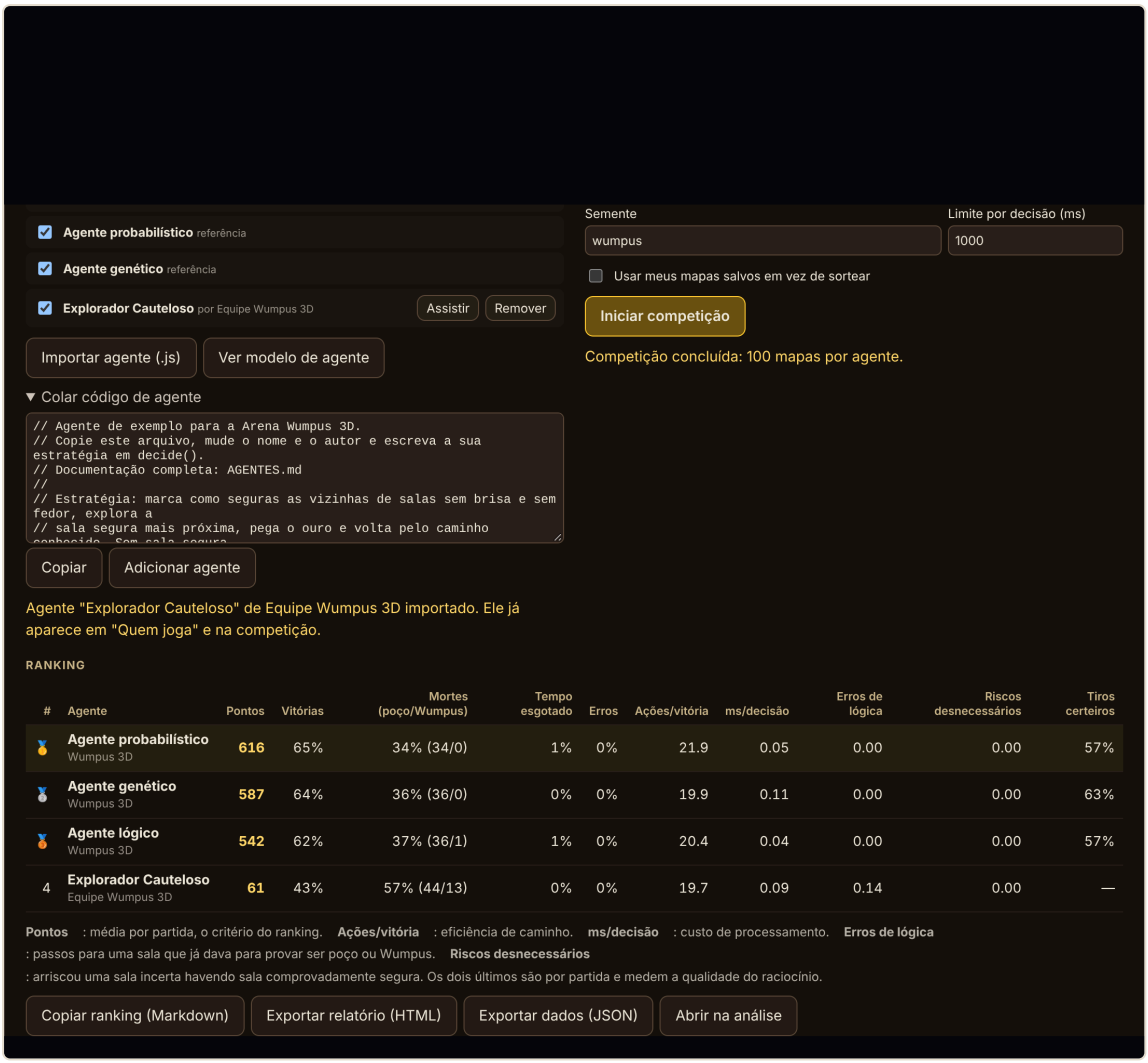
- **Erro de lógica**: entrar numa sala que já dava para *provar* ser poço ou Wumpus.
- **Risco desnecessário**: entrar numa sala incerta quando ainda existia sala comprovadamente segura para explorar.

Os agentes de referência têm zero nas duas métricas. O agente de exemplo tem alguns erros, porque a dedução dele é mais simples (não localiza o Wumpus).

## Executando e ranqueando (runner.ts e ranking.ts)

`playEpisode(controller, mapa, limite)` joga uma partida completa: pede a decisão (com prazo), consulta o árbitro, aplica a ação e mede o tempo. Exceções, ações inválidas e demoras viram `outcome: 'error'`, com pontuação de derrota. Tanto agentes embutidos quanto externos passam pela mesma interface `Controller`.

`aggregate()` junta as partidas de um agente em médias e `rank()` ordena: pontuação média, depois vitórias, depois menos erros de lógica.



Tela da Arena após uma competição de 100 mapas, com o agente de exemplo importado.

## Relatórios e Análise (src/report/)

Um **relatório** guarda as partidas de um ou mais agentes para comparar depois. Ele é gerado na **Avaliação** (um agente), na **Arena** (vários agentes nos mesmos mapas) e por `npm run arena -- --relatorio arquivo.html`.

src/report/report.ts — uma linha por partida — linhas 11–23

```
export interface GameRow {
  map: number; // número do mapa (1, 2, 3...): na Arena todos os agentes jogam o mesmo mapa n
  outcome: Outcome;
  score: number;
  actions: number;
  gold: boolean; // pegou o ouro
  killed: boolean; // matou o Wumpus
  shots: number;
  bumps: number;
  logicErrors: number;
  needlessRisks: number;
  ms: number; // tempo total de decisão
}
```

- `report.ts` não usa DOM: monta o relatório ( `buildReport` ), lê arquivos ( `parseReport` ) e calcula séries (média acumulada, contagem acumulada, média móvel). Por isso funciona no navegador e no Node.

- `charts.ts` gera os gráficos como **texto SVG**. As cores vêm de variáveis CSS (paleta categórica validada para daltonismo, nos temas claro e escuro), e `installChartHover` liga a linha vertical e a dica ao passar o mouse.
- `html.ts` monta a tabela-resumo, as barras de resultado, quatro gráficos por partida (pontuação média acumulada, vitórias acumuladas, mortes acumuladas e ações por partida) e a tabela partida a partida. `renderReportHtml` gera um arquivo independente, que abre em qualquer navegador sem internet.
- O HTML exportado leva os dados dentro de `<script type="application/json" id="wumpus3d-relatorio">`. A tela de Análise lê tanto esse HTML quanto o JSON puro, e aceita também a exportação antiga da Arena (só totais).

## Partidas humanas (src/report/human.ts)

Com "Você (teclado)" em "Quem joga", `src/ui/human-panel.ts` cria um `Referee` (o árbitro da Arena) para a partida e, quando ela termina, guarda um `HumanGame` no `localStorage` (`storage.ts`, até 1000 partidas). Cada registro leva o **mapa inteiro**: é isso que permite aos agentes jogarem depois exatamente os mesmos mapas.

src/report/human.ts — o que entra no relatório — linhas 52–60

```
export function countedGames(games: HumanGame[], player: string) {
  const seen = new Set<string>();
  return games.filter((g) => {
    if (g.player !== player || g.revealed) return false;
    const k = layoutKey(g.layout);
    if (seen.has(k)) return false;
    seen.add(k);
    return true;
  });
```

- **Contam** só a primeira vez que o jogador termina cada mapa (`layoutKey`), e só se ele não abriu o mapa (tecla M).
- **Desafio:** `challengeMap(semente, n, ...)` usa o mesmo sorteador da Arena (`arenaMaps`), então jogadores diferentes com a mesma semente recebem os mesmos mapas na mesma ordem. `nextChallengeIndex` só avança depois de uma partida terminada, para ninguém pular mapas.
- `compareWithAgents` joga os agentes probabilístico, lógico e genético nos mapas que o jogador terminou e monta um relatório com todos; no desafio, o número do mapa é a posição na semente, e relatórios de jogadores diferentes ficam alinhados mapa a mapa na Análise.

> **POR QUE UM HTML COM OS DADOS DENTRO**
>
> Um arquivo só serve para as duas coisas: qualquer pessoa abre e vê os gráficos, e o jogo importa o mesmo arquivo para comparar com outros. Não há servidor: nada sai do navegador.

> **COMPARANDO DE FORMA JUSTA**
>
> Na Avaliação, use a mesma **semente** e o mesmo número de partidas para todos os agentes: eles jogam exatamente os mesmos mapas, e o número do mapa (`map`) bate entre os relatórios. A tela de Análise avisa quando os relatórios usam mapas diferentes.

## Linha de comando e GitHub Actions

Exemplos

```
npm run arena -- agents/meu-agente.js --maps 200 --seed treino
npm run arena -- --maps 500 --ocultar-semente --json public/ranking.json
npm run arena -- --maps 200 --seed comparacao --relatorio relatorio.html
```

`npm run arena` compila `scripts/arena.ts` com o próprio Vite (modo SSR, para Node) e executa. Não precisa de nenhuma dependência extra.

- `.github/workflows/arena.yml`: em todo pull request que mexe em `agents/`, roda 200 mapas com uma semente nova e mostra o ranking no resumo da execução.
- `.github/workflows/deploy.yml`: a cada push na `main`, roda 500 mapas com a semente secreta `ARENA_SEED`, salva `public/ranking.json` e publica o site. A tela da Arena mostra esse "ranking oficial" se o arquivo existir.

> **POR QUE A SEMENTE OFICIAL É SECRETA**
>
> O motor é aberto e determinístico. Se a semente fosse pública, um dev poderia gerar os mapas da prova e programar um agente que "reconhece" cada mapa pelas primeiras percepções. Com a semente secreta, e uma semente nova a cada PR, isso não funciona.

# 10 A interface — src/ui, main.ts, index.html e style.css

`main.ts` só liga as peças: teclado, botões e painéis aos módulos de `src/ui`, um por responsabilidade. `src/main.ts` (140 linhas)

| MÓDULO | RESPONSABILIDADE |
|---|---|
| `state.ts` | O estado da partida em andamento ( `match` ) e os campos de configuração ( `cfg` ). |
| `game.ts` | Controle da partida: `act()` , `agentStep()` , `restart()` , câmera, mapa aberto e o laço `tick()` . |
| `hud.ts` | Placares, percepções, pensamento do agente, registro de eventos e tela de fim de jogo. |
| `players.ts` | Quem joga: nomes, opções do seletor e criação do agente (embutido ou importado). |
| `maps-panel.ts` , `genetic-panel.ts` , `bench-panel.ts` , `human-panel.ts` | Uma seção do painel cada: Salvar mapas, Agente genético, Avaliação e Minhas partidas. |
| `dom.ts` | Utilitários de tela usados também pela Arena e pela Análise: `$` , mensagens, download, copiar, confirmar com dois cliques. |

**POR QUE DIVIDIR O MAIN.TS (D21)**

O `main.ts` chegou a 799 linhas misturando estado, HUD, mapas, treino, avaliação e partidas humanas. Na refatoração de Código Limpo cada seção virou um módulo pequeno, com o estado compartilhado num só lugar ( `state.ts` ). O comportamento é o mesmo: um roteiro de uso no navegador deu resultados idênticos antes e depois.

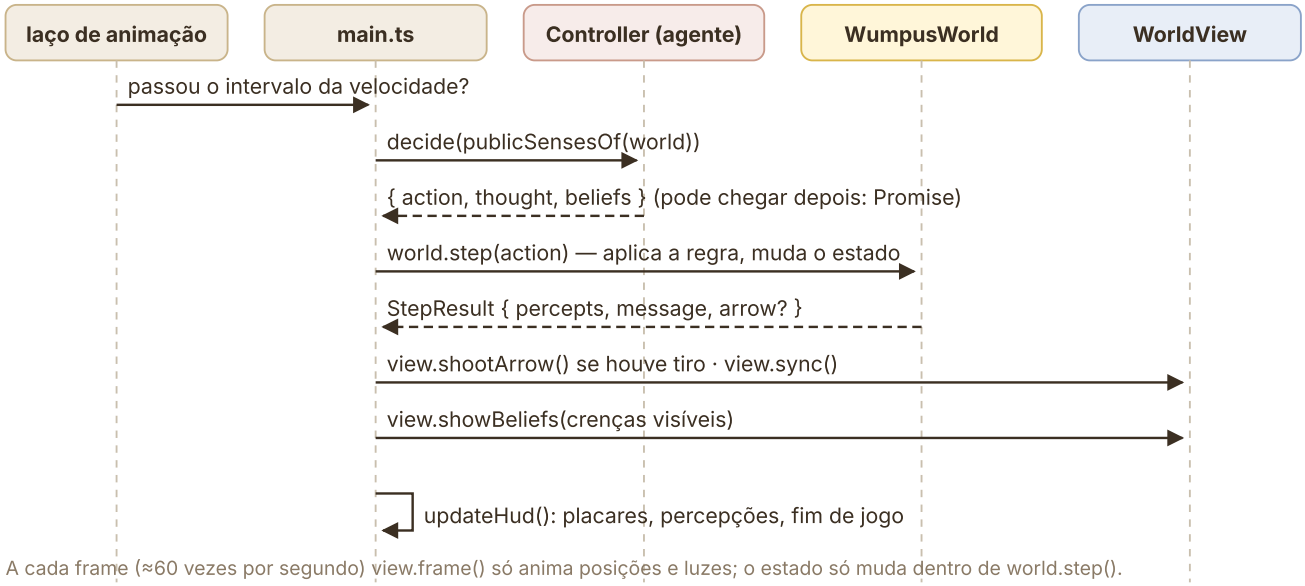
## O estado da partida

src/ui/state.ts — linhas 21–25

```
export interface Match {
  world: WumpusWorld;
  layout: MapLayout; // configuração do mapa atual, para repetir com outro jogador
  mapName: string | null; // nome do mapa salvo/importado em uso (null = sorteado)
  challenge: { seed: string; index: number } | null; // desafio por semente (D20)
```

`world` é a partida atual; `layout` , a configuração do mapa, para repetir a mesma partida com outro jogador; `controller` , quem joga ( `null` quando é você); `ready` , a promessa de que o agente terminou de se preparar; `decision` , o último pensamento e as crenças para mostrar; `game` , o contador de partidas explicado abaixo; `stepping` evita pedir uma decisão nova enquanto a anterior não chegou.

## Um passo do agente




A cada frame (≈60 vezes por segundo) view.frame() só anima posições e luzes; o estado só muda dentro de world.step().

src/ui/game.ts — agentStep() linhas 34–55

```
export async function agentStep() {
  const { controller, world } = match;
  if (!controller || world.gameOver || match.stepping) return;
  const game = match.game;
  const current = () => game === match.game;
  match.stepping = true;
  try {
    await match.ready;
    if (!current() || match.world.gameOver) return;
    const d = await controller.decide(publicSensesOf(match.world));
    if (!current()) return;
    match.decision = d;
    act(d.action);
  } catch (err) {
    if (!current()) return;
    match.playing = false;
    log(`Erro do agente: ${(err as Error)?.message ?? err}`);
    updateHud();
  } finally {
    if (current()) match.stepping = false;
  }
}
```

**ARMADILHA COMUM: RESPOSTAS ATRASADAS**

O agente externo responde por mensagem, então a resposta chega "depois". Imagine: o agente está pensando, você clica em **Parar** e a partida recomeça. Em seguida chega a resposta antiga e é aplicada à partida nova! O contador `game` resolve isso: cada partida tem um número, e uma resposta só é usada se o número ainda for o mesmo de quando foi pedida.

## Recomeçar: `restart(newMap)`

`restart()` sorteia um mapa novo; `restart(false)` repete o atual. Ao trocar de jogador o mapa é mantido, para comparar agentes no mesmo cenário. O botão ■ **Parar** (tecla T) chama `resetSameMap()`: interrompe o agente e volta ao início no mesmo mapa.

## O laço de animação

src/ui/game.ts — linhas 147–152

```
export function tick(now: number) {
  if (match.playing && match.controller && !match.world.gameOver && now - match.lastStep >= 1000 /
Number(cfg.speed.value)) {
    match.lastStep = now;
    agentStep();
  }
  view.frame();
```

O laço desenha um frame sempre e, se o agente estiver jogando, dá um passo quando passou o intervalo definido pela velocidade (passos por segundo).

## Layout (index.html e style.css)

- `#layout` é um flex com duas partes: `#stage` (o mapa 3D e os controles) e `#setup` (o painel de configuração, 340 px).
- Em telas com até 760 px de largura o layout vira coluna: mapa em cima, painel embaixo (45% da altura) e os botões numa barra entre os dois.
- Placares e histórico ficam dentro de `#viewport`, por cima do canvas, com `position: absolute`.
- A regra `[hidden] { display: none !important; }` garante que o atributo `hidden` esconda mesmo os elementos com `display: flex`.

> **POR QUE O PAINEL NÃO FLUTUA SOBRE O MAPA**
>
> Na primeira versão o painel ficava por cima do mapa e cobria parte dele. Agora ele divide a tela com o mapa, e o canvas 3D se redimensiona (via `ResizeObserver`) quando o painel abre ou fecha.



No celular (390 px): mapa em cima e botões numa barra; o painel abre embaixo.

## Armazenamento (storage.ts)

| CHAVE DO LOCALSTORAGE | CONTEÚDO |
|---|---|
| `wumpus3d.maps` | Lista de mapas salvos ( `MapLayout` ). |
| `wumpus3d.genome` | Pesos treinados do agente genético. |
| `wumpus3d.agents` | Agentes importados (nome, autor, descrição e código). |

Toda leitura e escrita está dentro de `try/catch`. Em janela anônima, ou com o armazenamento bloqueado, o jogo continua funcionando e só não guarda nada. Mapas lidos do armazenamento passam de novo por `parseLayout`.

> **O QUE A INTERFACE NÃO FAZ**
>
> - Não sincroniza nada entre dispositivos: o `localStorage` é de cada navegador. Para levar um mapa ou agente, exporte o arquivo.
> - Não tem tradução: os textos estão em português, direto no HTML e no código.
> - Não tem testes automatizados de interface (os testes do capítulo 13 cobrem regras, deduções e Arena).

# 11 Mapas salvos, exportação e importação

O fluxo completo, de ponta a ponta:

1. **Salvar**: `layout` (o mapa atual) recebe um nome e vai para `saveMap()`. Um mapa com o mesmo nome é substituído.
2. **Jogar um salvo**: `playLayout(l)` troca o `layout` e chama `restart(false)`.
3. **Exportar**: o JSON é baixado como arquivo e também colocado numa caixa de texto. Em ambientes que bloqueiam downloads, dá para copiar dali.
4. **Importar**: o texto (de arquivo ou colado) passa por `parseLayout()`; se for válido, é salvo e carregado; se não, a mensagem de erro explica o problema.
5. **Excluir**: pede um segundo clique ("Confirmar?"), porque `confirm()` nem sempre está disponível.

src/ui/maps-panel.ts — importJson() linhas 47–57

```
function importJson(text: string) {
  try {
    const l = parseLayout(text);
    const saved = saveMap(l);
    refreshMapList(l.name);
    playLayout(l);
    mapMsg(saved ? `Mapa "${l.name}" importado e salvo.` : `Mapa "${l.name}" carregado (este navegador
não permite salvar).`);
  } catch (err) {
    mapMsg(`Não deu para importar: ${(err as Error).message}`, true);
  }
}
```

# 12 Roteiro de reimplementação

Uma ordem sugerida para reconstruir o projeto do zero. Cada etapa produz algo que funciona e pode ser testado antes de seguir. Não pule a verificação de cada etapa: um erro nas regras contamina tudo que vem depois.

> **ORDEM IMPORTA**
>
> As etapas 2, 3, 8 e 10 são as mais sensíveis. Escreva testes automatizados para elas (capítulo 13) antes de seguir. Depois que a interface estiver pronta, é muito mais difícil descobrir se um erro está nas regras ou no desenho.

| ETAPA | O QUE CONSTRUIR | PRONTO QUANDO... |
|---|---|---|
| 1 | Projeto Vite + TypeScript ( `npm create vite@latest -- --template vanilla-ts` ) e instalar `three` . | `npm run dev` abre uma página. |
| 2 | `world.ts` : tipos, `key()` , geração de mapa, `sense()` e `step()` . | Num script, você joga uma partida no console, vê as percepções mudarem e consegue morrer e vencer. |
| 3 | Geração com BFS (só mapas vencíveis). | Em 10.000 mapas, nenhum tem ouro em poço ou sem caminho. |
| 4 | Cena 3D estática: piso, paredes, câmera, luz. | A grade aparece no navegador. |
| 5 | `WorldView.load()` e `sync()` : desenhar o estado do mundo. | Poços e Wumpus aparecem nas posições certas com "revelar mapa". |
| 6 | Teclado, botões e placares ( `main.ts` , `src/ui/game.ts` , `updateHud` ). | Você joga uma partida completa no 3D. |
| 7 | Animações em `frame()` (interpolação, tocha, partículas, flecha). | O agente desliza entre as salas em vez de pular. |
| 8 | `Knowledge` : `pitFree` , `wumpusCandidates` , `certainPits` , `safe` . | Nos exemplos do livro, as salas seguras batem com as do AIMA. |
| 9 | Agente lógico e laço de jogo automático (velocidade, um passo). | O agente vence a maioria dos mapas 4×4 sem nunca dar erro de lógica. |
| 10 | Probabilidades ( `pitProbabilities` ) e agente probabilístico. | O exemplo do capítulo 7 dá 56%. |
| 11 | Mostrar crenças no 3D e o pensamento do agente. | Os discos e rótulos aparecem na fronteira. |
| 12 | Mapas: `MapLayout` , `parseLayout` , salvar, exportar, importar. | Exportar e importar devolve exatamente o mesmo mapa. |
| 13 | Agente genético, algoritmo genético e Web Worker de treino. | O treino partindo de pesos aleatórios melhora a aptidão em poucas gerações. |
| 14 | Arena: `protocol` , `sandbox` , `runner` , `referee` , `ranking` , `rng` . | Agentes com laço infinito, sintaxe errada ou ação inválida geram erro claro sem travar a página. |
| 15 | Tela da Arena e linha de comando. | Mesma semente dá o mesmo ranking no navegador e no terminal. |
| 16 | Workflows do GitHub e publicação. | O site abre no GitHub Pages com o ranking oficial. |

# 13 Como o projeto foi verificado

A versão documentada foi verificada com scripts executados durante o desenvolvimento (fora do repositório):

- **Regras**: partidas roteirizadas conferindo que voltar à entrada sem ouro não encerra o jogo e que voltar com ouro dá vitória com 1487 pontos.
- **Geração**: 10.000 mapas sorteados, todos vencíveis.
- **Agentes**: centenas a milhares de partidas por agente e tamanho, nos mesmos mapas (tabela do capítulo 8).
- **Navegador**: testes com Playwright (navegador automatizado) para jogar até a vitória, salvar/exportar/importar mapas, treinar o genético, parar e reiniciar, importar agentes defeituosos e rodar a Arena, em tamanhos de computador e de celular, sempre conferindo que não houve erro no console.
- **Arena**: agentes de teste com laço infinito, erro de sintaxe, ação inválida, exceção no meio da partida e tentativa de acessar o `process` do Node. Todos foram tratados com mensagem clara.

## Testes automatizados

O repositório tem uma suíte de testes com o Vitest, que roda com `npm test` (ou `npm run check`, que também confere os tipos). O GitHub Actions roda os testes antes de publicar o site, e uma falha impede a publicação.

| ARQUIVO | O QUE GARANTE |
|---|---|
| `tests/world.test.ts` | Regras: vitória só com o ouro na entrada, morte, Wumpus morto, limite de ações, custos, fedor depois da morte, mapas sorteados sempre vencíveis, exportar/importar e validação de mapas inválidos. |
| `tests/knowledge.test.ts` | Deduções: exemplo de 56% do AIMA, poço certo, localização do Wumpus, flecha que erra, Wumpus fora de poço deduzido e P(Wumpus) ponderada. |
| `tests/arena.test.ts` | Arena: mesma semente gera os mesmos mapas, agentes de referência sem erro de lógica, lógico vence o aleatório, decisão inválida vira erro claro. |
| `tests/genetic.test.ts` | Agente genético: pesos que dependem da posição não o fazem andar em círculos; o `BAYES_LIKE_GENOME` não esgota o tempo. |
| `tests/human.test.ts` | Partidas humanas: só a primeira vez em cada mapa e sem mapa aberto contam, desafio reproduzível que só avança depois de terminar, agentes jogam os mesmos mapas do jogador. |
| `tests/report.test.ts` | Relatórios: totais batem com as partidas, o HTML exportado é importado de volta igual, exportação antiga da Arena é aceita, arquivos estranhos são recusados. |

tests/world.test.ts — exemplo linhas 10–23

```
describe('vitória e fim de partida', () => {
  it('voltar à entrada sem o ouro não encerra a partida', () => {
    const w = WumpusWorld.fromLayout(layout());
    play(w, ['forward', 'left', 'left', 'forward']);
    expect(w.status).toBe('playing');
  });

  it('pegar o ouro e voltar à entrada dá vitória com +1000 e +500', () => {
    const w = WumpusWorld.fromLayout(layout());
    play(w, ['forward', 'forward', 'grab', 'left', 'left', 'forward', 'forward']);
    expect(w.status).toBe('won');
    expect(w.score).toBe(SCORE.gold + SCORE.escape - 7);
  });
```

**O QUE OS TESTES NÃO COBREM**

A interface (botões, painel, Arena na tela) e o 3D não têm testes automatizados no repositório. Eles foram verificados com Playwright durante o desenvolvimento, mas esses scripts não foram incluídos.

# 14 Limitações conhecidas e próximos passos

| LIMITAÇÃO | POR QUE EXISTE / O QUE FAZER |
|---|---|
| Sem testes automatizados de interface | As regras, as deduções e a Arena têm testes (capítulo 13); a tela e o 3D não. Testes com Playwright seriam o próximo passo. |
| Probabilidade do Wumpus calculada sala a sala | Usa P(sem poço) de cada candidata, sem enumerar poços e Wumpus juntos. Uma enumeração conjunta seria exata e mais lenta. |
| O agente probabilístico olha só o risco imediato | Escolhe a sala de menor risco agora, sem pesar quanto ela ajuda a achar o ouro ou o valor da informação que traria. Uma decisão por utilidade esperada (teoria da decisão) poderia ir além. |
| Estimativa aproximada com mais de 16 salas na fronteira | Para não travar em mapas grandes. Poderia dividir a fronteira em grupos independentes e enumerar cada grupo. |
| Os agentes supõem poços independentes | O mapa tem um número fixo de poços (que os agentes não conhecem, por escolha) e o gerador descarta mapas invencíveis; as probabilidades reais diferem um pouco das calculadas. |
| Fedor continua depois que o Wumpus morre | Regra do livro. A base de conhecimento já ignora fedor quando `wumpusDead` é verdadeiro. |
| Memória de vídeo não é liberada ao recomeçar | `level.clear()` remove objetos, mas não chama `dispose()` nas geometrias e materiais. Em sessões muito longas a memória cresce. Corrija percorrendo a cena e chamando `dispose()`. |
| Pacote de ~600 KB (160 KB compactado) | `import * as THREE` traz a biblioteca inteira. Importar só as classes usadas pode reduzir. |
| O genético ganha pouco do probabilístico no 4×4 | Com as mesmas informações, a regra "menor risco" já está perto do melhor possível em mapas pequenos. Novas características (por exemplo, valor da informação ou distância até o ouro provável) poderiam mudar isso. |
| O `vm` do Node não é uma barreira de segurança | Revise o código dos agentes antes do merge. Para mais isolamento, rode a arena num contêiner sem rede. |
| Agentes externos com o mesmo nome e autor se substituem | O id é derivado de autor/nome. Use um id explícito se precisar de versões lado a lado. |
| Dados só no navegador | O `localStorage` não sincroniza. Um backend seria necessário para contas e ranking em tempo real. |
| Acessibilidade parcial | Botões têm foco visível e textos, mas o mapa 3D não tem descrição textual para leitores de tela. |

# 15 Glossário

| TERMO | SIGNIFICADO |
|---|---|
| Agente | Programa que percebe o ambiente e escolhe ações. |
| Percepção | O que o agente sente na sala atual (brisa, fedor, brilho, batida, grito). |
| Base de conhecimento | Tudo que o agente anotou e deduziu (classe `Knowledge`). |
| Fronteira | Salas desconhecidas vizinhas de salas já visitadas: os próximos lugares possíveis. |
| Probabilidade a priori | A chance antes de qualquer evidência (20% de poço por sala). |
| Inferência bayesiana | Atualizar probabilidades com base no que foi observado. |
| Enumeração | Listar todas as combinações possíveis e somar as que são consistentes. |
| BFS (busca em largura) | Busca que explora por camadas de distância; acha o caminho mais curto num grid. |
| Genoma | A lista de pesos que define a estratégia do agente genético. |
| Aptidão (fitness) | A nota de um genoma: pontuação média nas partidas. |
| Elitismo | Os melhores passam sem alteração para a próxima geração. |
| Seleção por torneio | Sortear alguns indivíduos e escolher o melhor como pai. |
| Mutação gaussiana | Somar um ruído aleatório com distribuição normal a um peso. |
| Validação | Avaliar em dados (mapas) que não foram usados no treino. |
| Web Worker | Código JavaScript rodando numa thread separada da página. |
| Sandbox | Ambiente isolado para rodar código de terceiros. |
| Semente (seed) | Valor inicial de um sorteador; a mesma semente gera os mesmos números. |
| Determinístico | Que dá sempre o mesmo resultado para a mesma entrada. |
| Interpolação (lerp) | Andar uma fração do caminho entre dois valores a cada frame. |
| Harness | Código fixo que embrulha o código do dev e fala com a arena. |
| Promise | Valor que vai chegar no futuro (resposta assíncrona). |
| Gerador (`function*`) | Função que entrega valores aos poucos com `yield`. |